Infinite-ISP Reference Model v2.0.0

Description of Algorithms

29th October 2026

About this Document:

Purpose

This document is a supplement provided with the Infinite-ISP Reference Model, it provides algorithm details of all modules in the ISP as well as the explanation of configuration parameters.

Revision History

Version	Released Date	Change Description
1.00	2023-06-01	Initial Draft – Reference Model V1
1.01	2023-09-09	Added Sharpening Module
1.01	2026-10-23	Added Local dynamic contrast improvement Module
2.0.0	2026-10-29	Added Auto Exposure Statistics, EV-law Auto Exposure, and Saturation Enhancement modules

Contents

List of Figures **4**

List of Tables **5**

Introduction **6**

Motivation **7**

Modules Description **8**

Crop . . . 9
Dead Pixel Correction . . . 10
Algorithm Explanation . . . 10
Configuration Parameters . . . 10
Black Level Correction . . . 12
Algorithm Explanation . . . 12
Configuration Parameters . . . 12
Opto-Electronic Conversion Function - OECF . . . 14
Algorithm Explanation . . . 14
Configuration Parameters . . . 14
Bayer Noise Reduction . . . 15
Algorithm Explanation . . . 15
Configuration Parameters . . . 15
Digital Gain . . . 17
Algorithm Explanation . . . 17
Configuration Parameters . . . 17
White Balance . . . 18
Algorithm Explanation: . . . 18
Configuration Parameters . . . 18
3A STATS . . . 19
Auto White Balance . . . 20
Algorithm Explanation . . . 20
Configuration Parameters . . . 20
Auto Exposure Statistics . . . 22
Configuration Parameters . . . 22
Auto Exposure . . . 23
Algorithm Explanation . . . 23
Configuration Parameters . . . 28
Color Filter Array . . . 30
Algorithm Explanation . . . 30
Configuration Parameters . . . 35
Color Correction Matrix . . . 36
Algorithm Explanation . . . 36
Configuration Parameters . . . 36
Gamma Correction . . . 37
Algorithm Explanation . . . 37
Configuration Parameters . . . 37
Color Space Conversion . . . 38
Algorithm Explanation . . . 38
Configuration Parameters . . . 39
Local Dynamic Contrast Improvement . . . 40
Algorithm Explanation . . . 40

Configuration Parameters 40
Sharpening 42
Algorithm Explanation 42
Configuration Parameters 42
2D Noise Reduction 43
Algorithm Explanation 43
Configuration Parameters 43
Saturation Enhancement 45
Configuration Parameters 45
RGB Conversion 46
Configuration Parameters 46
Invalid Region Crop 47
Configuration Parameters 47
Scale 48
Algorithm Explanation 48
Configuration Parameters 48
YUV Format - 444-422 49
Algorithm Explanation 49
Configuration Parameters 50

Pipeline Results **52**

References **54**

List of Figures

Figure 1 : Infinite-ISP Reference Model pipeline 6
Figure 2 : Defective and corrected pixels 10
Figure 3 : BNR algorithm 15
Figure 4 : Auto Exposure control loop 23
Figure 5 : AE metering: (a) default grid weights, (b) clip-corrected cell mean 24
Figure 6 : AE decision: (a) deadband, damping and slew limit, (b) highlight and clip limits 26
Figure 7 : AE response to two scene changes (reference model, default configuration) 27
Figure 8 : AE actuation: choosing the nearest gain on an EV scale 27
Figure 9 : AE gain oscillation handling between two adjacent gains 28
Figure 10 : Flowchart for the Malwar He Cutler's Demosaicing Algorithm 31
Figure 11 : Filter Coefficients for linear interpolation of R, G & B data. 32
Figure 12 : Raw Image with RGGB Bayer pattern 32
Figure 13 : Masking channels 32
Figure 14 : Convolution of a raw image with Filter Type 1 33
Figure 15 : Estimation of g at r and b locations 33
Figure 16 : Final G channel 33
Figure 17 : Location of R pixels masks 33
Figure 18 : Convolution of Type 2 filters with raw image for R estimated images 34
Figure 19 : Extracting estimated R values 34
Figure 20 : Final R Channel 35
Figure 21 : Pictorial representation of 4:4:4 and 4:2:2 49
Figure 22 : Memory view of bytes for packed format 49
Figure 23 : 4:4:4 format 50
Figure 24 : 4:2:2 format 50
Figure 25 : Memory view of 4:2:2 50
Figure 26 : Subsample entries for 4:2:2 51
Figure 27 : Pipeline Results 53

List of Tables

Table 1 : Crop configuration parameters . . . 9
Table 2 : DPC configuration parameters . . . 11
Table 3 : BLC configuration parameters . . . 12
Table 4 : OECF configuration parameters . . . 14
Table 5 : BNR configuration parameters . . . 15
Table 6 : DG configuration parameters . . . 17
Table 7 : WB configuration parameters . . . 18
Table 8 : AWB configuration parameters . . . 20
Table 9 : AES configuration parameters . . . 22
Table 10 : AE configuration parameters . . . 28
Table 11 : CCM configuration parameters . . . 36
Table 12 : GC configuration parameters . . . 37
Table 13 : CSC configuration parameters . . . 39
Table 14 : LDCI configuration parameters . . . 41
Table 15 : Sharpening Configuration Parameters . . . 42
Table 16 : 2DNR configuration parameters . . . 43
Table 17 : SE configuration parameters . . . 45
Table 18 : RGBC configuration parameters . . . 46
Table 19 : IRC configuration parameters . . . 47
Table 20 : Scaling – Valid Output Sizes . . . 48
Table 21 : Scale configuration parameters . . . 48
Table 22 : YUV Formats configuration parameters . . . 51
Table 23 : IQ metrics of the reference-model output against ground truth . . . 53

Introduction

Infinite-ISP Reference Model (aka Model) is a collection of 21 Python modules which convert an input RAW image from a sensor to an output YUV image. All the modules are individually configurable via various parameters. Some of the module's configurable parameters are determined by using a separate software tuning tool while others are more directly determined. The model pipeline is shown in Figure 1.

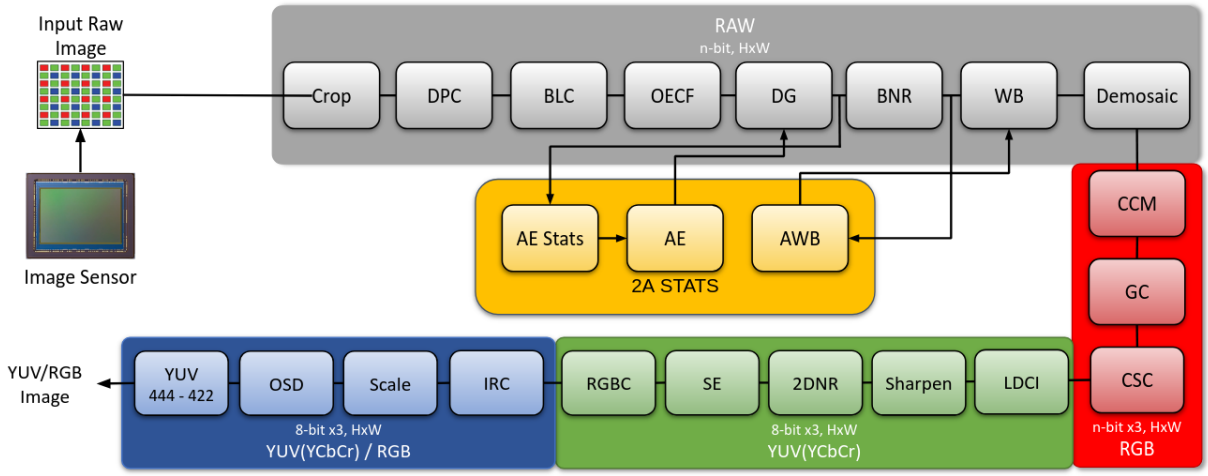

Figure 1: Infinite-ISP Reference Model pipeline

Motivation

The real motivation behind Infinite-ISP is to streamline procedures, enhance current approaches, and develop advance algorithms in the ISP and computational photography field. This project provides a platform for relevant developers to work on the most exciting and challenging hardware-related issues that an ISP engineer encounters. The "open innovation" approach, enables anyone with the right skills, time, and interest to contribute generously to the Infinite-ISP project.

Modules Description

Currently the model consists of the following 21 modules,

- Crop
- Dead Pixel Correction (DPC)
- Black Level Compensation (BLC)
- Optoelectronic Conversion Function (OECF)
- Digital Gain (DG)
- Bayer Noise Reduction (BNR)
- White Balance (WB)
- Auto White Balance (AWB)
- Demosaic
- Color Correction Matrix (CCM)
- Gamma Correction (GC)
- Auto Exposure (AE)
- Color Space Conversion (CSC)
- Local Dynamic Contrast Improvement
- Sharpening
- 2d Noise Reduction (2DNR)
- Saturation Enhancement (SE)
- RGB Conversion (RGBC)
- Invalid Region Crop (IRC)
- Scale
- YUV Conversion Format (YUV 444 - 422)

Crop

The crop module is used to extract a rectangular region from an image by selecting an area at the center of the image with a specific size. This is useful for focusing on a specific portion of the image or more importantly removing unwanted regions from the edges.

Algorithm Explanation

A straightforward cropping algorithm has been implemented, ensuring the preservation of the Bayer pattern. This is achieved by verifying that the number of rows and columns to be cropped are divisible by 4, and subsequently cropping an equal number of pixels from all sides of the image.

Cropping is applied on image only when both of the following conditions are met:

- Parameters [new_height, new_width] provided to the crop module are even numbers
- The difference between image dimensions and crop parameters is divisible by 4:

$$(image_height - new_height) \ mod \ 4 = 0$$

$$(image_width - new_width) \ mod \ 4 = 0$$

Configuration Parameters

Table 1: Crop configuration parameters

Parameters	Details
is_enable	When enabled it crops the image if the conditions listed above are met: False: Disable True: Enable
new_width	New width of the input RAW image after cropping
new_height	New height of the input RAW image after cropping

Dead Pixel Correction

Image sensors sometimes come with manufacturing flaws that result in some malfunctioning pixels in the image. Despite advanced manufacturing techniques photo sensor arrays like CCD and CMOS sensor arrays include defective pixels because of noise, dust, or fabrication flaws. Hence identifying the defective pixels in the images and then replacing them with approximate values are two important phases of the DPC module.

Defective pixels can be categorized into two groups:

1. Dead: consistently low output
2. Hot: consistently high output

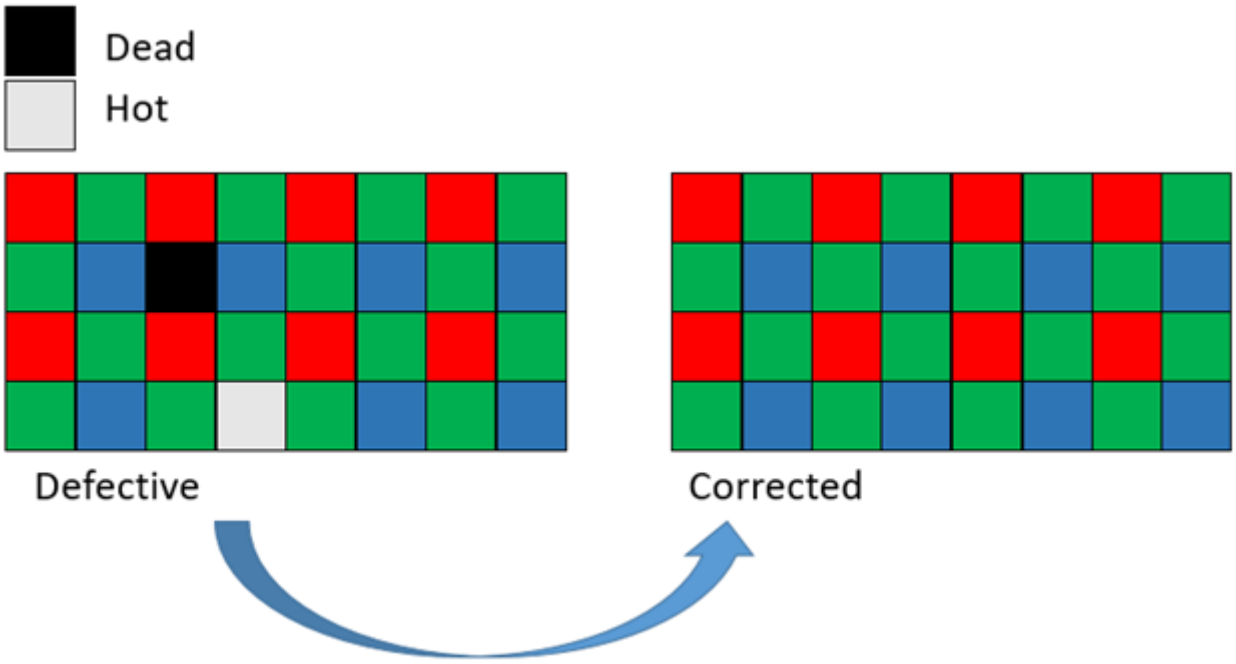

Figure 2: Defective and corrected pixels

Algorithm Explanation

Each pixel of the image is tested by the DPC algorithm. To find defective pixels a 3×3 neighborhood of the *same* color channel surrounding the pixel being tested in a 5×5 window is created. The pixel is considered defective if it meets both of the following conditions:

1. The pixel value is less than the minimum pixel value of the all the pixels in the 3×3 neighborhood or greater than the maximum pixel value of all the pixels in the 3×3 neighborhood.
2. The difference between the pixel's value and each one of the pixels in the 3 x 3 neighborhood is greater than the programmed threshold parameter.

Once the pixel is identified as defective, its value is corrected by first computing four gradients (horizontal, vertical, left diagonal, and right diagonal) which pass through the pixel. The corrected value is then computed as the average of the two neighbors in the *minimum* gradient direction.

Configuration Parameters

Table 2: DPC configuration parameters

Parameters	**Details**
is_enable	When enabled, apply the DPC algorithm: False: Disable True: Enable
dp_threshold	The threshold used for determining defective pixels. A low threshold results in more pixels being detected as dead and hence corrected

Black Level Correction

This module is responsible for setting the image's pure black color. Setting the exposure time and various programmable gains (analog and digital) to their minimum values is one of the techniques to create a pure black image; however, even under these conditions, the resulting image may not be entirely black and as such the black level needs to be corrected. The four offset parameters, one for each channel, needed by this algorithm are generated by using the Tuning tool for a specific image sensor. Once the parameters have been generated, they are used by this module to correct the image's black level.

Algorithm Explanation

To apply the black level correction, subtract the corresponding parameters from each of the pixels in the image. For a RGGB Bayer image the new values are computed as follows:

$$R' = R - r_offset$$

$$Gr' = Gr - gr_offset$$

$$Gb' = Gb - gb_offset$$

$$B' = B - b_offset$$

Additionally, by applying these offsets the range of values that a pixel generates is scaled down. This can optionally be corrected (enabled via the is_linear parameter) by applying a linearization function over the desired range of values. Black level correction with linearization is computed as follows:

$$R' = \frac{R - r_offset}{r_sat - r_offset}\ 2^n$$

$$Gr' = \frac{Gr - gr_offset}{gr_sat - gr_offset}\ 2^n$$

$$Gb' = \frac{Gb - gb_offset}{gb_sat - gb_offset}\ 2^n$$

$$B' = \frac{B - b_offset}{b_sat - b_offset}\ 2^n$$

Configuration Parameters

Table 3: BLC configuration parameters

Parameters	Details
is_enable	When enabled, apply the BLC algorithm: False: Disable True: Enable
r_offset	Red channel offset
gr_offset	Gr channel offset

Parameters	Details
gb_offset	Gb channel offset
b_offset	Blue channel offset
is_linear	Enables or disables linearization. When enabled a linearization function is applied to scale the values over the desired range of values
r_sat	Red channel saturation level
gr_sat	Gr channel saturation level
gb_sat	Gb channel saturation level
b_sat	Blue channel saturation level

Opto-Electronic Conversion Function - OECF

The relationship between the sensor output (photocell voltages) and incident light lux is called the opto-electronic conversion function (OECF). The sensor usually has an approximately linear response in that the detected signal is proportional to the incident light lux as imaged by the lens. Linearity is essential for achieving good white balance and color accuracy. The OECF module implements lookup curves for voltage re-mapping for a *specific* sensor. There are four lookup curves, one for each color channel which are generated through the Tuning tool.

Algorithm Explanation

The Tuning tool uses ISO 14524 to generate the four curves. It uses a digital reflective camera contrast chart to plot density response vs. pixel voltage values.

Configuration Parameters

Table 4: OECF configuration parameters

Parameters	Details
is_enable	When enabled, applies the OECF curve: False: Disable True: Enable
r_lut	The lookup table for the OECF curve. This curve is sensor dependent and is calculated by the Tuning tool

Bayer Noise Reduction

The Bayer Noise Reduction (BNR) block suppresses noise in the Bayer domain before demosaicing. As a noise reduction block in the ISP pipeline, it is primarily tasked with reducing noise without effectively blurring the image details.

Algorithm Explanation

For the BNR block, a Joint/Cross Bilateral Filter (JBF) has been selected as the appropriate candidate for RAW image denoising.

1. Green channel interpolation is performed on the input Bayer image to obtain a complete image of the green pixels using green interpolation kernels derived from the Malwar-He-Cutler algorithm.
2. Input Bayer image is deconstructed into R and B sub-images of half the size compared to the original Bayer image.
3. Interpolated G image based on a window (size determined by the file window parameter) is deconstructed into G pixels just at the R pixel locations. Additionally, the interpolated G image is also deconstructed into G pixels just at the B pixel locations. This produces R and B guided images.
4. JBF is applied on the R and B sub-images (produced in step 2) using their respective guide images (produced in step 3). JBF is also applied on the interpolated G image using itself as the guide image.
5. JBF uses two types of kernels i.e. range and spatial. A look-up table weighing scheme is created for the range kernel to facilitate the algorithm's hardware implementation.
6. The output Bayer image is reconstructed by joining the R, G, and B pixels from the JBF outputs (step 4).

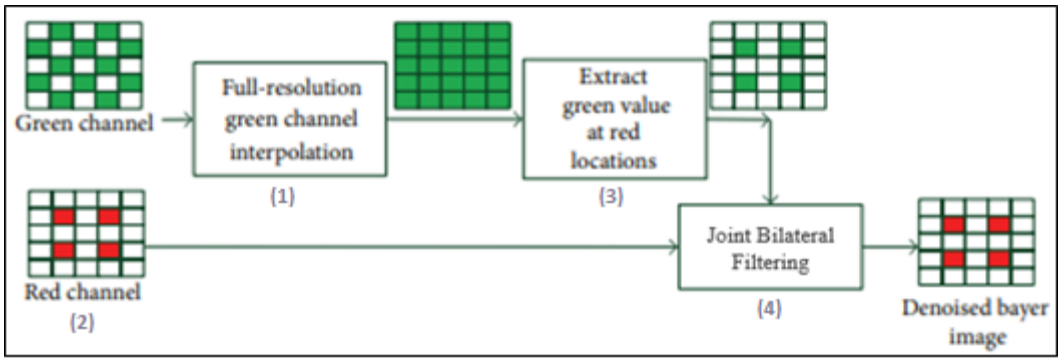

Figure 3: BNR algorithm

Configuration Parameters

Table 5: BNR configuration parameters

Parameters	Details
is_enable	When enabled, applies the BNR algorithm: False: Disable True: Enable
filt_window	Filter size for BNR This should be an odd window size e.g., 3 x 3, 5x5, etc.

Parameters	Details
r_std_dev_s	Red channel Gaussian kernel strength Higher strength values will result in increased blurring while a value of 0 is not permissible
r_std_dev_r	Blue channel range kernel strength Higher strength values will result in increased blurring while a value of 0 is not permissible
g_std_dev_s	Gr and Gb Gaussian kernel strength. Higher strength values will result in increased blurring while a value of 0 is not permissible
g_std_dev_r	Gr and Gb range kernel strength Higher strength values will result in increased blurring while a value of 0 is not permissible
b_std_dev_s	Blue channel Gaussian kernel strength Higher strength values will result in increased blurring while a value of 0 is not permissible
b_std_dev_r	Blue channel range kernel strength Higher strength values will result in increased blurring while a value of 0 is not permissible

Digital Gain

Digital gain works with the auto exposure module to choose an appropriate constant value to be multiplied to all the channels to improve the image exposure.

Algorithm Explanation

In the digital gain module, the raw image I is multiplied by an integer g, also known as Gain.

$$I' = I * g$$

$$g = gain_array[current_gain]$$

An array of permissible gains, gain_array, is defined in the configuration, and the index of the applied gain is given by the current_gain parameter. In auto mode (is_auto), current_gain is chosen by the Auto Exposure control block, which selects the gain index that brings the metered image brightness to the target exposure (see the Auto Exposure section); otherwise it is taken directly from the configuration. The Digital Gain module itself only applies $gain_array[current_gain]$ to all channels and clips the result to the sensor bit depth.

Configuration Parameters

Table 6: DG configuration parameters

Parameters	Details
is_auto	Flag to adjust digital gain according to AE Feedback False : current_gain can only be changed manually True : current_gain is adjusted according to ae_feedback
gain_*array*	List of permissible gains
current_gain	Index of the gain_array for the current gain which is being used. The index value starts at 0
ae_feedback	AE feedback parameter 0 : Correct Exposure 1 : Overexposed -1 : Underexposed

White Balance

This module adjusts the white balance of raw images by applying appropriate gains, ensuring that white colors appear true to life. If gains are not available, they must first be determined through an auto-white balance process.

Algorithm Explanation:

The white balance module selectively applies the provided gains to the red and blue channels, while the green channel remains unaffected. This process effectively restores color balance, making white colors appear accurate and natural.

$$R' = R * r_gain$$

$$B' = B * b_gain$$

Configuration Parameters

Table 7: WB configuration parameters

Parameters	Details
is_enable	Applies user-given white balance gains when enabled: False: Disable True: Enable
is_auto	Flag to adjust white balance gains according to AWB Algorithm 0 : r_gain and b_gain are applied on the raw image 1 : Gains from AWB module are applied
r_gain	Red channel gain.
b_gain	Blue channel gain.

3A STATS

The 3A Stats module encompasses three essential aspects of the ISP (Image Signal Processing) pipeline: Auto White Balance (AWB), Auto Focus (AF), and Auto Exposure (AE). These 3A algorithms do not modify the image directly, but instead provide feedback in the form of 3A statistical parameters which are used by other pipeline modules. The AF module will be implemented in the future.

Auto White Balance

AWB computes white balance gains as 3A statistics using the Grey World algorithm. In the White Balance (WB) module the AWB calculated gains are applied to the raw image.

Algorithm Explanation

The AWB module calculates gains that adjust the color balance in an image to restore the accurate colors of grey pixels. The Grey World algorithm is chosen for its optimal balance between ease of implementation in RTL (Register-Transfer Level) and overall performance.

There are two main steps of the AWB module:

Preprocess Pixel Filtering

2. Gain Calculation using Grey World Algorithm

Before calculating white balance gains, a pre-processing pixel filtering step is performed, in which a percentage of underexposed and overexposed pixels are removed from the image. The percentage of underexposed and overexposed pixels is provided through AWB configuration parameters (underexposed_percentage, overexposed_percentage). These pixels are not used in the next step.

Grey World Algorithm The Grey World algorithm is a white balance method based on the assumption that, on average, a scene's color is neutral grey. This assumption holds when there is a balanced distribution of colors in the scene. Given this balanced distribution, the average reflected color represents the color of the light source. To estimate the color cast of the illumination, the algorithm compares the average color to grey. The Grey World algorithm calculates an illumination estimate by computing the mean of each image channel

$$C_{avg} = \frac{\sum_{n=0}^{N} C(n)}{N}$$

$C \in \{R,\ G.\ B\}$ and N is the total number of pixels in the image. White balance gains for red and blue channels are calculated as shown below. These gains are then applied to the corresponding channels of the next frame.

$$R_{gain} = \frac{G_{avg}}{R_{avg}}$$

$$B_{gain} = \frac{G_{avg}}{B_{avg}}$$

Configuration Parameters

Table 8: AWB configuration parameters

Parameters	Details
is_enable	Calculate white balance gains of the image when enabled: False: Disable True: Enable
stats_window_offset	Specifies crop dimensions to obtain a stats calculation window Should be an array of element, [Up, Down, Left, Right] Should be a multiple of 4

underexposed_percentage	[0 - 100] - Set % of dark (underexposed) pixels to exclude before AWB gain calculation
overexposed_percentage	[0 - 100] - Set % of saturated (overexposed) pixels to exclude before AWB gain calculation

Auto Exposure Statistics

Auto Exposure Statistics is a bit-exact fixed-point model of the AE statistics block. It measures the raw frame after digital gain. The measurement is taken over a 3x3 grid of cells whose boundaries are 16-bit registers.

For each cell it computes four quantities, all held as 32-bit counters. The first is a green-pixel sum. The second is the count of green pixels that fall below a shadow threshold. The third is the count of green pixels that rise above a highlight threshold. The fourth is the count of pixels, on any channel, that are clipped to 0 or to the sensor's full scale. Auto Exposure later uses the green-pixel sum, together with the cell's green-pixel count, to derive a clip-corrected mean brightness.

The per-cell counters wrap on overflow, like the hardware registers they model. The green sum is the one exception: it saturates at its 32-bit maximum instead. If it wrapped, a bright cell would read as dark and would steer the exposure control the wrong way.

This module only measures the frame. It performs no exposure decision. The Auto Exposure module decides what to do with these statistics.

Configuration Parameters

Table 9: AES configuration parameters

Parameters	**Details**
is_enable	When enabled, computes the AE statistics. Must be enabled for Auto Exposure to run.
col_starts	4 ascending column boundaries of the grid (16-bit registers), in pixels, the last closing the grid
row_starts	4 ascending row boundaries of the grid (16-bit registers), in pixels
shadow_threshold	Green pixels below it are counted as shadows (DN at the sensor bit depth)
highlight_threshold	Green pixels above it are counted as highlights (DN at the sensor bit depth)

Auto Exposure

Auto Exposure (AE) is one of the 3A control modules. It uses the AE statistics of the current frame to choose the digital gain for the next frame, so that the image reaches a target brightness without clipping its highlights. In the model, digital gain stands in for the sensor exposure that AE controls on a camera.

AE works in the exposure value (EV) domain, where 1 EV is a factor of two in brightness, and holds all EV quantities in Q8 fixed point (256 = 1 EV). The control runs once per frame, after the statistics are computed, and the selected gain is applied to the following frame, so AE always acts one frame behind what it measured.

Figure 4 shows the complete loop: the statistics are computed on the raw image after digital gain, the AE control turns them into a new gain index, and that gain is applied to the next frame.

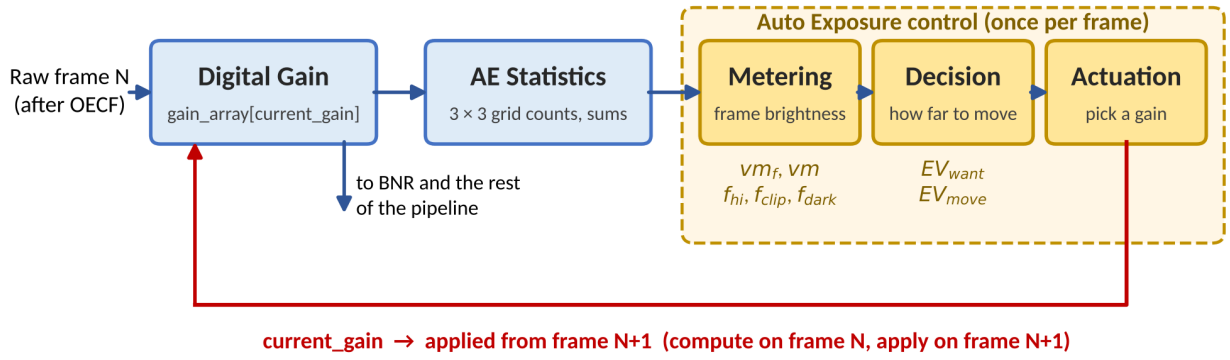

Figure 4: Auto Exposure control loop

Algorithm Explanation

Each frame, the AE control performs three steps: metering, decision and actuation.

Metering

The metering step measures the brightness of the raw current frame from the AE statistics. Brightness is estimated using the green pixels only.

The frame is first divided into a 3×3 grid. For each cell, AE computes the average green value, excluding pixels that are clipped to full black or full white, since these no longer carry brightness information. The statistics count clipped pixels across all color channels, so AE estimates the green share as half of each count. If too few valid green pixels remain in a cell (fewer than valid_min_pct percent), the cell is skipped as unreliable.

For cell i, the estimated clipped greens, the valid greens and the clip-corrected mean are:

$$w_i = \min\left(\left\lfloor \frac{WC_i}{2} \right\rfloor, H_i\right), \qquad b_i = \min\left(\left\lfloor \frac{BC_i}{2} \right\rfloor, S_i\right)$$

$$v_i = g_i - (w_i + b_i)$$

$$m_i = \frac{M_i - w_i\,(2^N - 1)}{v_i}$$

where M_i is the sum of the green pixel values in the cell, g_i is the number of green pixels in the cell (known from the grid boundaries), WC_i and BC_i are the white and black clip counts, H_i and S_i are the highlight and shadow counts, and N is the sensor bit depth, so that the w_i white-clipped greens at full scale are removed from the sum.

The cell averages are then combined into a single weighted mean for the frame, using the per-cell weights ω_i set by grid_weights. By default the center cell has the largest weight, since the subject is usually in the middle of the frame:

$$vm = \frac{\sum_i \omega_i \, m_i}{\sum_i \omega_i}$$

To keep the exposure from reacting to small frame-to-frame changes caused by noise, this mean is smoothed over time with a first-order IIR filter, whose strength α is set by vm_iir_a (in percent). The filter is bypassed when the same frame is processed repeatedly in render_3a mode:

$$vm_f \leftarrow vm_f + \frac{\alpha}{100}\left(vm - vm_f\right)$$

In addition, AE computes three frame-level fractions, in per mille, that describe how much of the frame lies at its extremes: green pixels above the highlight threshold, green pixels below the shadow threshold, and pixels of any channel clipped to full white. Here n_i is the total number of pixels in cell i:

$$f_{hi} = 1000 \cdot \frac{\sum_i H_i}{\sum_i g_i}, \qquad f_{dark} = 1000 \cdot \frac{\sum_i S_i}{\sum_i g_i}, \qquad f_{clip} = 1000 \cdot \frac{\sum_i WC_i}{\sum_i n_i}$$

Figure 5 shows the default grid weights and how the clipped pixels of a cell are left out of its average.

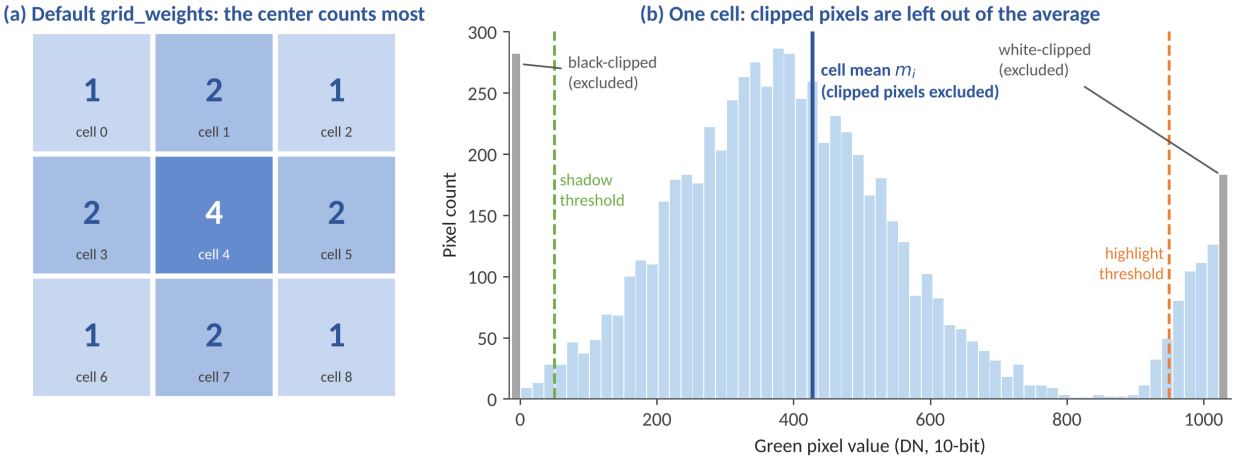

Figure 5: AE metering: (a) default grid weights, (b) clip-corrected cell mean

The metering step passes the following values to the decision step:

- vm_f **(filtered metered mean):** the smoothed, center-weighted green mean of the frame. The exposure error is computed from it.
- vm **(raw metered mean):** the unfiltered frame mean. The actuation step uses it to detect a scene change while a gain is being held.
- f_{hi} **(highlight fraction):** per mille of green pixels above highlight_threshold. It is compared with the hi_frac_pm budget to protect highlights.
- f_{clip} **(clip fraction):** per mille of pixels at full scale. It is compared with the clip_frac_pm budget to limit clipping.
- f_{dark} **(dark fraction):** per mille of green pixels below shadow_threshold. It is compared with dark_frac_pm so that a mostly dark frame is never made darker.
- **Valid flag:** whether at least one cell could be metered. If none could, the decision step uses a fixed step instead of the error.

Decision

The decision step uses the metering results to work out how much the exposure should change for the next frame. It works in the exposure value (EV) domain, where 1 EV is a factor of two in brightness, and holds all EV quantities in Q8 fixed point (256 = 1 EV).

AE first computes the exposure error as the difference, in EV, between the target brightness target_mean and the filtered metered mean vm_f. A positive error means the frame is too dark, and a negative error means it is too bright:

$$EV_{err} = \log_2\big(\text{target_mean}\big) - \log_2\big(vm_f\big)$$

The error is then adjusted to protect the image at its extremes. If the highlight fraction f_{hi} or the clip fraction f_{clip} exceeds its budget (hi_frac_pm or clip_frac_pm), the requested correction is pulled down so that highlights are not burned out. The strength of each pull is set by hi_k and clip_k, and together they can lower the request by at most ev_max_pull:

$$EV_{hi} = -\frac{\text{hi_k}}{100}\Big(\log_2 f_{hi} - \log_2(\text{hi_frac_pm})\Big), \qquad EV_{clip} = -\frac{\text{clip_k}}{100}\Big(\log_2 f_{clip} - \log_2(\text{clip_frac_pm})\Big)$$

$$EV_{want} = \max\Big(\min\big(EV_{err},\, EV_{hi},\, EV_{clip}\big),\; EV_{err} - \text{ev_max_pull}\Big)$$

Each limit is zero when its fraction equals its budget. Below the budget it is positive, so it can only restrict how much brighter AE may go; above the budget it becomes negative and makes the exposure darker. A limit is applied only when its fraction is non-zero.

In the opposite direction, if the dark fraction f_{dark} exceeds dark_frac_pm, AE never asks for a darker exposure, because the frame is already mostly dark:

$$EV_{want} = \max\big(EV_{want},\, 0\big) \quad \text{if } f_{dark} > \text{dark_frac_pm}$$

If metering found no valid cell, AE cannot trust the mean. Instead it requests a fixed correction of ev_slew_max, darker if the clip fraction outweighs the dark fraction and brighter otherwise.

Finally, AE decides whether to move at all, and by how much. If the requested correction is within the deadband ev_deadband, the exposure is considered correct and nothing changes. Once the loop has converged, an extra hysteresis margin ev_hyst is added, so small fluctuations do not restart it. Here $c = 1$ when the loop has converged and 0 otherwise:

$$\big|EV_{want}\big| \le \text{ev_deadband} + c \cdot \text{ev_hyst} \quad \Longrightarrow \quad \text{no move}$$

Outside the deadband, AE corrects only part of the error per frame, set by the damping d (ev_damp, in percent), and limits each step to at most s (ev_slew_max), so the exposure changes smoothly instead of jumping. A non-zero request always moves by at least the smallest Q8 step:

$$EV_{move} = \min\Big(\max\Big(EV_{want} \cdot \frac{d}{100},\; -s\Big),\; s\Big)$$

Figure 6 shows how a requested correction is turned into a move (a), and how the highlight and clip limits depend on their fractions (b), using the default configuration.

The decision step passes the following values to the actuation step:

- EV_{want} **(requested correction):** the full exposure correction after the highlight, clip and dark adjustments. The actuation step uses it to judge whether stepping to a neighboring gain brings the exposure closer to the target.
- EV_{move} **(move for this frame):** the damped, slew-limited correction to apply now. A value of 0 means the exposure is held.

The actuation step also reuses three values from metering: the raw mean vm, to detect a scene change while a gain is held, and the fractions f_{hi} and f_{clip}, to check the highlight and clip budgets when choosing between two gains.

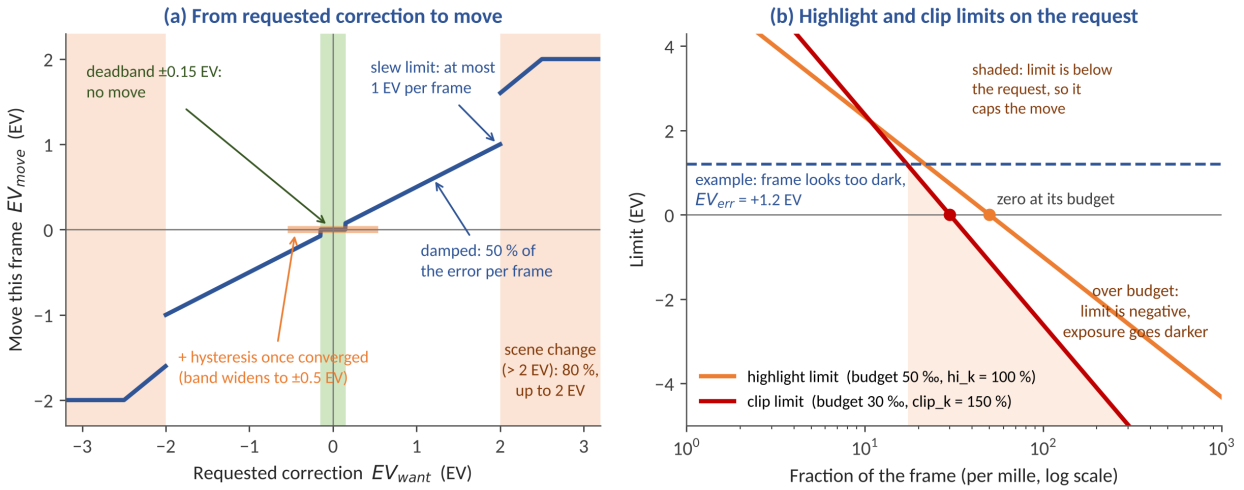

Figure 6: AE decision: (a) deadband, damping and slew limit, (b) highlight and clip limits

Scene Change Handling

A large exposure error usually means the scene itself has changed, for example when the camera moves from indoors to outdoors. In that case, the normal damping and slew limit would make AE take many frames to catch up. So when the magnitude of the requested correction EV_{want} is larger than scene_change_ev, AE switches to the faster settings: ev_damp_fast instead of ev_damp, and ev_slew_fast instead of ev_slew_max. Each frame then corrects a larger share of the error in a bigger step, so AE adapts quickly to the new scene. Once the error falls back below scene_change_ev, the normal settings apply again:

$$(d,\, s) = \begin{cases} (\text{ev_damp_fast},\ \text{ev_slew_fast}) & \text{if } |EV_{want}| > \text{scene_change_ev} \\ (\text{ev_damp},\ \text{ev_slew_max}) & \text{otherwise} \end{cases}$$

Figure 7 shows the AE code of the reference model running on a synthetic scene with the default configuration. AE first converges from a very dark start, then follows a 10× brighter and a 4.4× darker scene. On the first bright frame every cell is clipped, so there is no valid reading and AE requests a fixed −1 EV step; once readings are valid again, the fast settings bring the exposure back within a few frames.

Actuation

The actuation step turns the requested exposure change into an actual digital gain for the next frame. AE can only choose from the gains listed in the digital gain table gain_array, so it has to find the entry that best matches the request.

To compare gains with the request, each gain in the table is converted to EV (its $\log_2$). AE adds the move EV_{move} to the EV of the current gain to get the target exposure, then selects the table entry closest to that target:

$$k^* = \arg\min_k \left| \log_2(g_k) - \left(\log_2(g_{cur}) + EV_{move} \right) \right|$$

If the closest entry is the current gain, meaning the move was too small to reach another entry, AE still steps to the neighboring gain when that brings the exposure closer to the full requested correction EV_{want}. This keeps the loop from getting stuck short of the target.

Figure 8 shows an example: from a current gain of 3, a move of +0.62 EV lands closest to the gain 5 entry.

The actuation step passes the index of the selected gain, current_gain, to the Digital Gain module, and that gain is applied from the next frame.

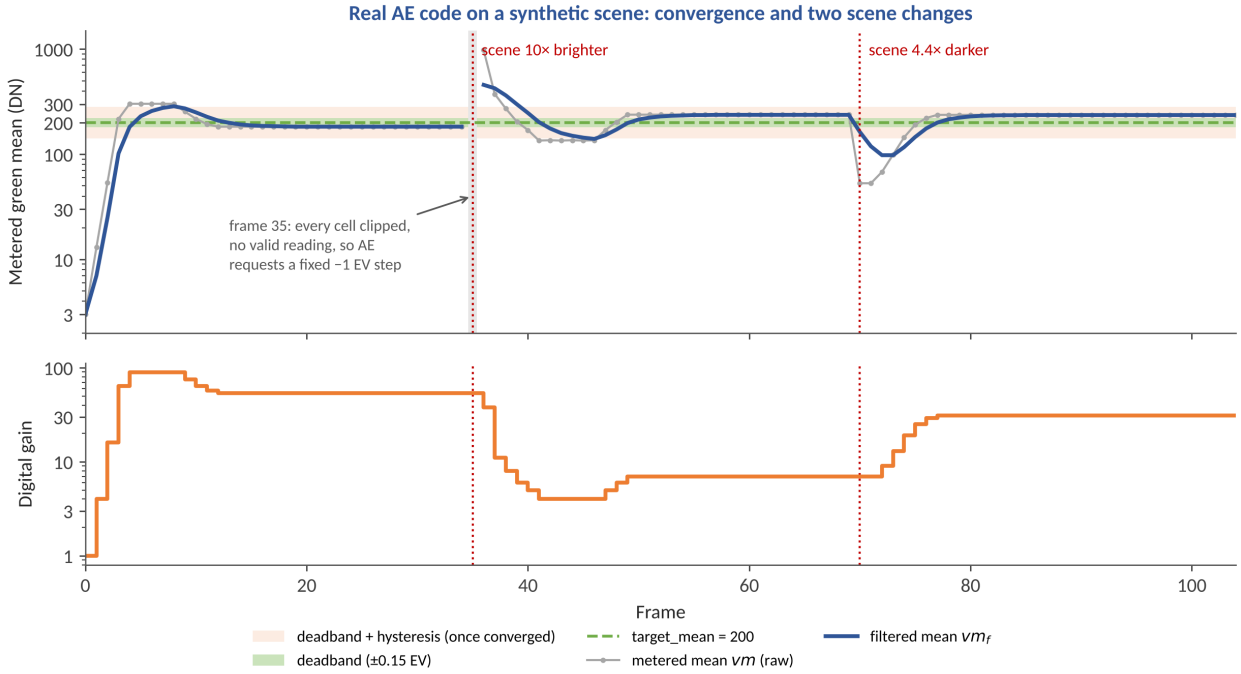

Figure 7: AE response to two scene changes (reference model, default configuration)

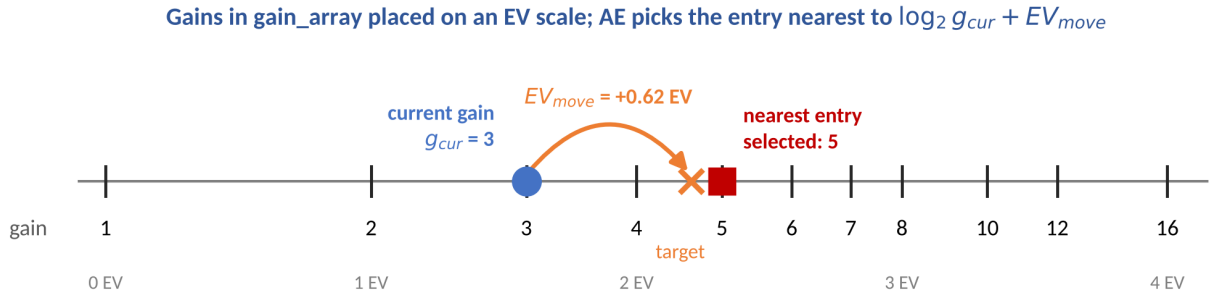

Figure 8: AE actuation: choosing the nearest gain on an EV scale

Gain Oscillation Handling

The gain table is coarse compared with a camera's exposure control, so the target brightness can fall between two adjacent gains. In that case, one gain makes the frame slightly too dark and the next makes it slightly too bright, and the loop would keep switching between them every few frames. AE detects this back-and-forth and settles on one of the two gains: the one with the smaller exposure error, or the darker one on a tie. It never settles on the brighter gain if that gain would exceed the highlight (hi_frac_pm) or clip (clip_frac_pm) budget. AE then holds that gain until the raw metered mean vm moves by more than ev_deadband from its value vm_{held} at the held gain, which means the scene has changed, and normal control resumes:

$$\left|\log_2(vm) - \log_2(vm_{held})\right| \leq \text{ev_deadband} \quad \Longrightarrow \quad \text{keep the held gain}$$

Figure 9 shows this with the AE code of the reference model, using a gain table in 1 EV steps and hysteresis turned off. The exposure needed lies between gains 2 and 4; after one trip to gain 4, AE returns to gain 2, which has the smaller error, and holds it.

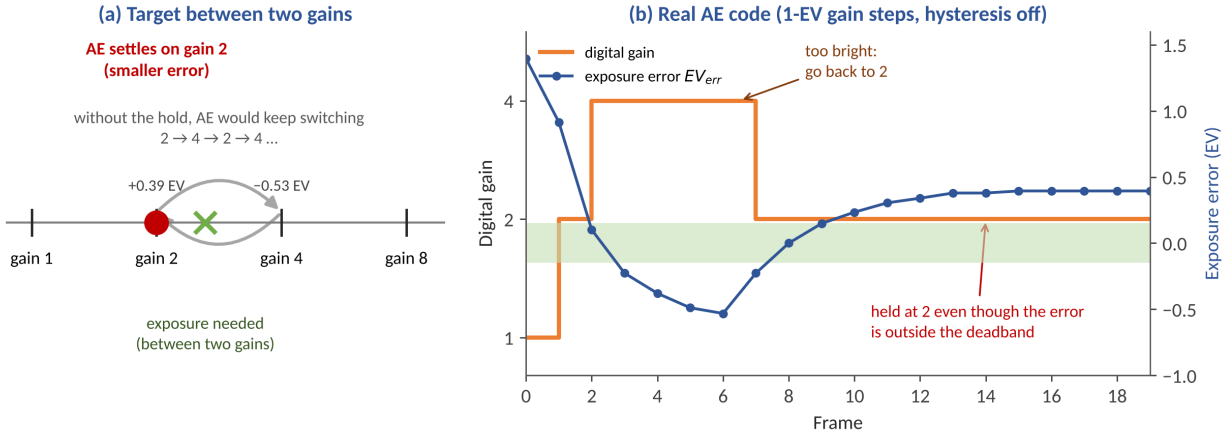

Figure 9: AE gain oscillation handling between two adjacent gains

Configuration Parameters

Table 10: AE configuration parameters

Parameters	Details
is_enable	When enabled, applies the Auto Exposure control. Requires auto_exposure_stats.is_enable and digital_gain.is_auto to be True. False: Disable; True: Enable
target_mean	[1, $2^{\text{bit_depth}} - 1$] – Target metered green mean, in DN at the sensor bit depth
valid_min_pct	[0, 100] – Minimum percentage of unclipped green pixels for a grid cell to be metered
grid_weights	9 values, each [0, 255] – Metering weight of each grid cell (cell index = 3 × row + column)
ev_deadband	[0, 1023] – Deadband in Q8 EV; errors within it do not change the gain

Parameters	Details
ev_hyst	[0, 1023] – Additional deadband in Q8 EV once the loop has converged
ev_damp	[0, 100] – Percentage of the exposure error corrected per frame
ev_damp_fast	[0, 100] – Percentage of the exposure error corrected per frame on a scene change
ev_slew_max	[1, 4095] – Largest move per frame, in Q8 EV
ev_slew_fast	[1, 4095] – Largest move per frame on a scene change, in Q8 EV
scene_change_ev	[0, 4095] – An exposure error above this value (Q8 EV) is treated as a scene change
vm_iir_a	[1, 100] – IIR filter strength (%) on the metered mean between frames; bypassed in render_3a
hi_frac_pm	[0, 1000] – Highlight budget: green pixels above highlight_threshold, per mille
hi_k	[0, 255] – Strength (%) of the highlight limit
clip_frac_pm	[0, 1000] – Clip budget: pixels at full scale, per mille
clip_k	[0, 255] – Strength (%) of the clip limit
dark_frac_pm	[0, 1000] – Above this shadow fraction (per mille), the frame is never made darker
ev_max_pull	[0, 4095] – Maximum amount (Q8 EV) by which the highlight and clip limits can lower the request

Color Filter Array

The CFA module is an integral component of the ISP pipeline that transforms the black-level corrected, white-balanced, linearized 2D image into a 3D RGB image format.

A Color Filter Array (CFA) is positioned in front of the sensor to capture color information. This array typically made of three filters, restricts the sensitivity of each photocell to one part of the visible spectrum. As a result, each pixel of the CFA image only contains information about this limited range, i.e., one color response. However, to render images on a specific display device, three colors per pixel are required. Demosaicing is therefore employed to reconstruct the missing colors at each pixel location.

Algorithm Explanation

After thorough analysis, Malvar-He-Cutler demosaicing algorithm was selected (shown in Figure 10). Image demosaicing, or demosaicing, is the interpolation problem of estimating complete color information for an image captured through a Color Filter Array (CFA), particularly using the Bayer pattern. Demosaicing is achieved by convolving the image with a set of linear filters. There are eight different filters shown in Figure 11 for interpolating the various color components at different locations.

The CFA module is implemented in the pipeline after DPC, BLC, and WB. Instead of using a constant or near-constant hue approach, the algorithm uses the criterion of edges to have much stronger luminance than chrominance components.

In the Malvar-He-Cutler algorithm, to interpolate a green value at a red pixel location, the algorithm compares the red pixel information with its estimate of a bilinear interpolation of the nearest red samples. If it differs from that estimate, it means there is a sharp luminance change at that pixel. Consequently, the algorithm bilinearly interpolates the green value by adding a portion of this estimated luminance change. This method is a gradient-corrected bilinear interpolation approach and its flowchart is shown below:

The Malvar-He-Cutler demosaicing algorithm is explained below:

1. **Mask Generation:** From a raw Bayer image shown in Figure 12 three r, g, and b masks are generated shown in Figure 13.

G Channel Extraction

1. On G channel, g pixels at r and b locations are estimated using the result of the convolution of filters from type 1 in Figure 11 with the raw image as shown below.
2. Result of step 1 is masked through r and b masks to get g pixels value at r and b location as shown below:
3. Interpolated g values of Step 2 are added at the respective r and b location to obtain a final G channel:

R Channel Extraction

1. Identify three locations where R has to be estimated as shown below, essentially creating three masks:
2. Convolve the raw image with each of the three types 2 filters shown in Figure 11 to produce three R estimated images.
3. Use the estimated R images produced in Step 2 along with the masks produced in Step 1 to get the estimated R channel values.
4. Estimated R channels values from step 3 together with R channel values of raw image produce the final R channel.

B Channel Extraction

For B channel extraction the same procedure as for the R channel is followed. At Step 1 estimating locations are now B at Gb, B at Gr and B at R where B channel has to be estimated and filters from type 3 will be applied in Step 2.

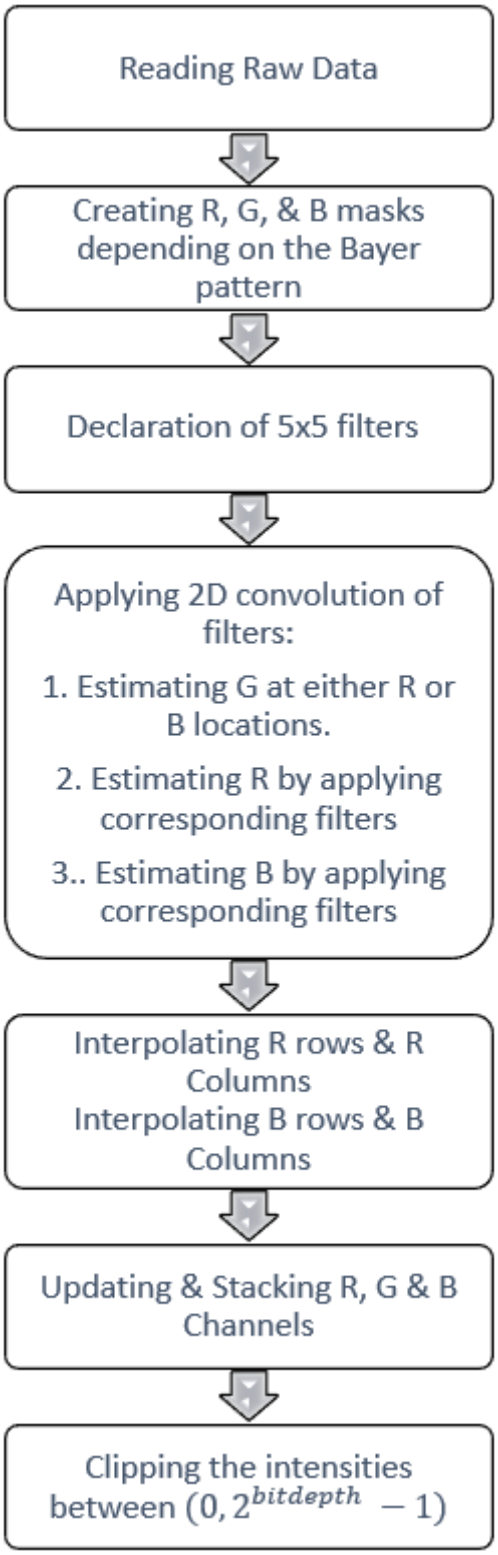

Figure 10: Flowchart for the Malwar He Cutler's Demosaicing Algorithm

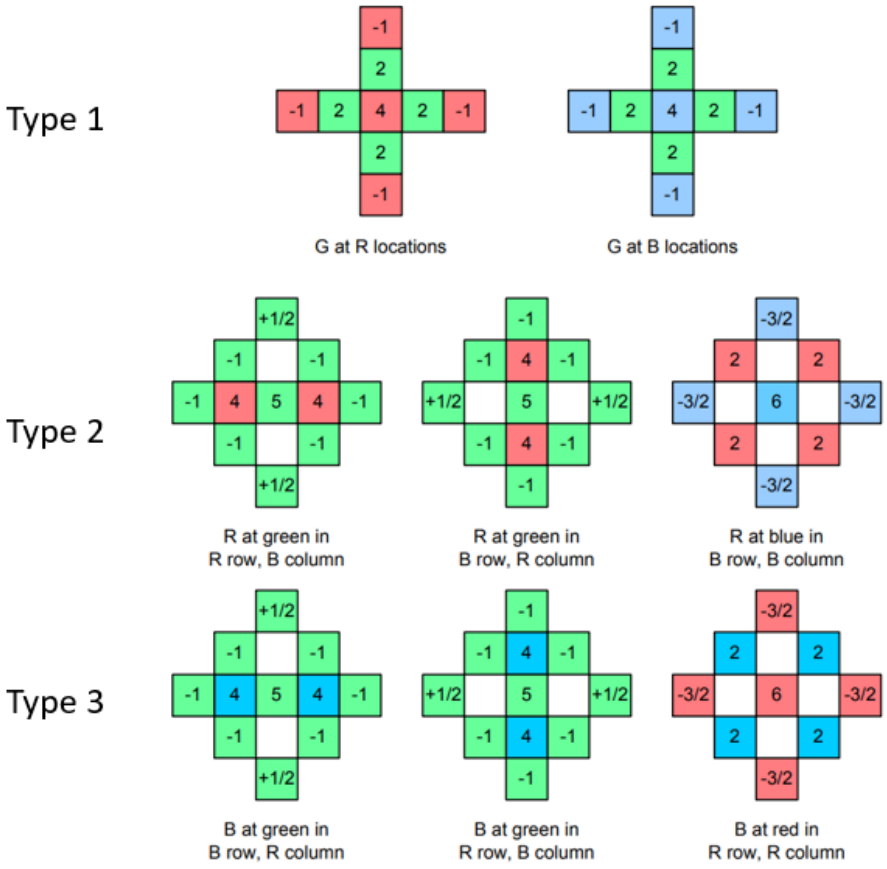

Figure 11: Filter Coefficients for linear interpolation of R, G & B data.

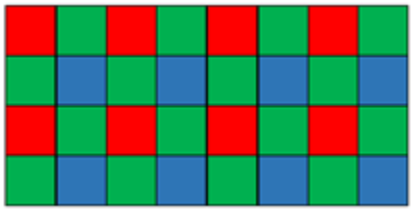

Figure 12: Raw Image with RGGB Bayer pattern

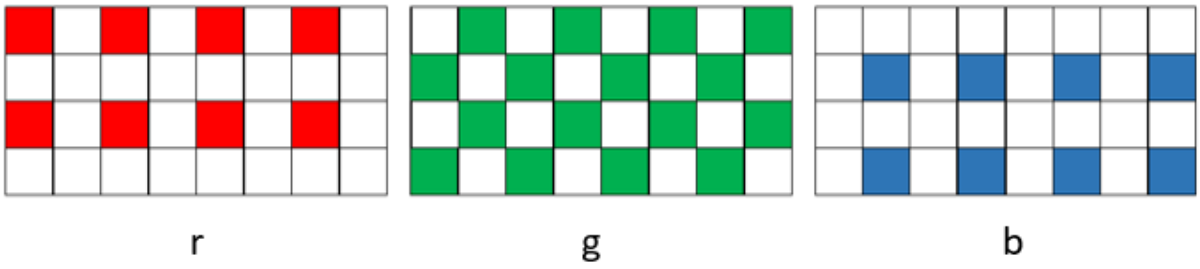

Figure 13: Masking channels

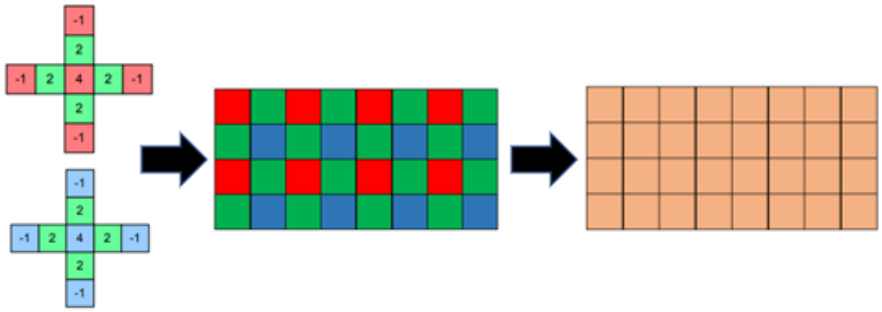

Figure 14: Convolution of a raw image with Filter Type 1

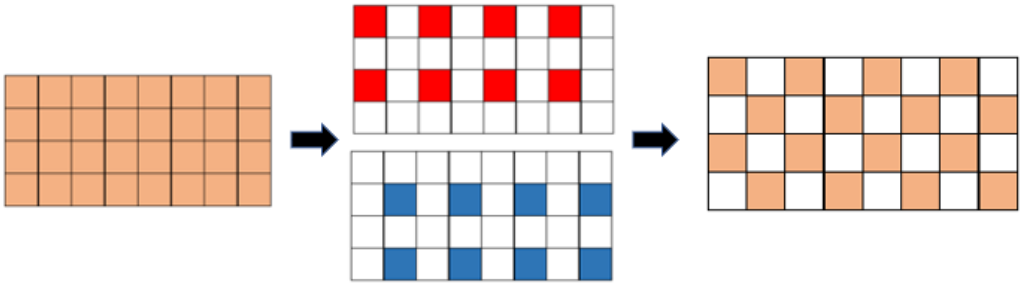

Figure 15: Estimation of g at r and b locations

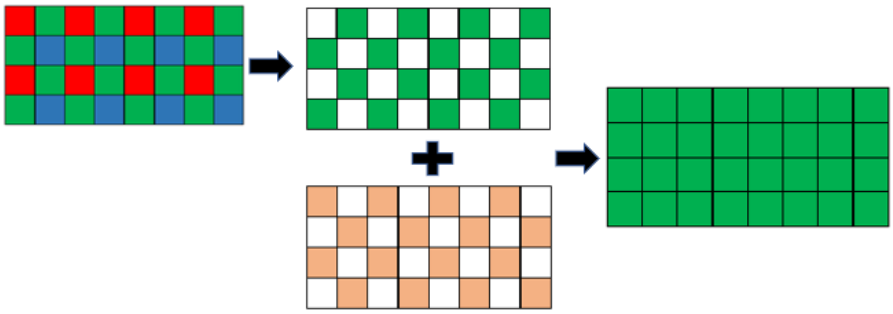

Figure 16: Final G channel

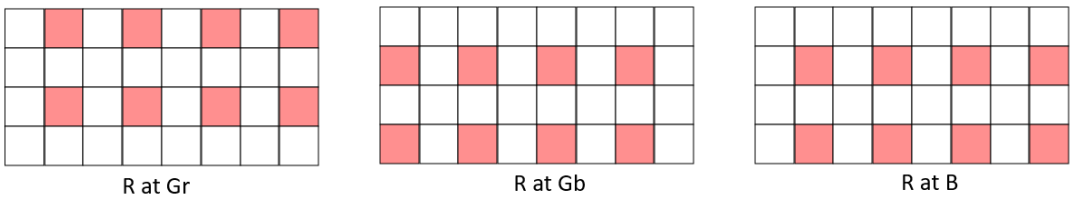

Figure 17: Location of R pixels masks

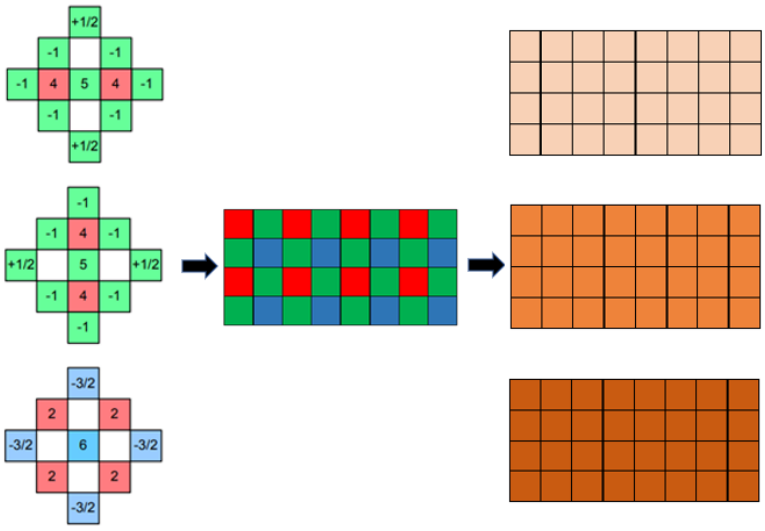

Figure 18: Convolution of Type 2 filters with raw image for R estimated images

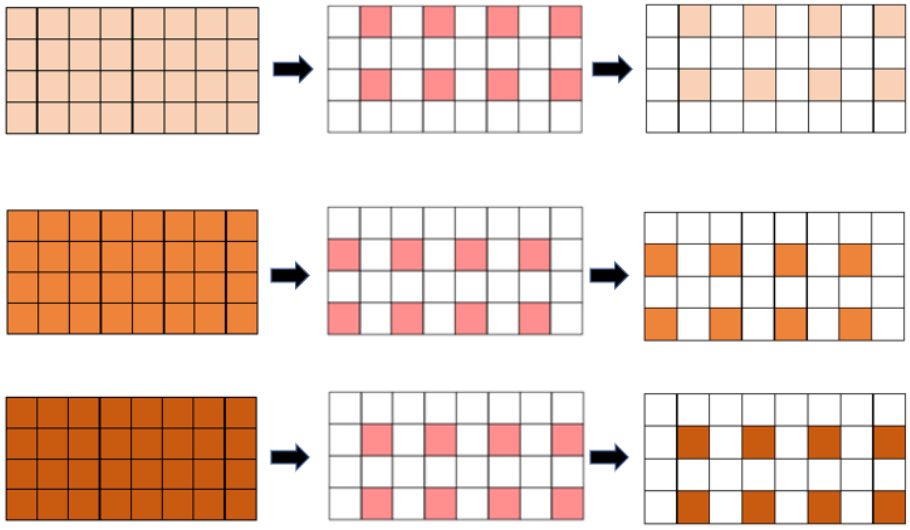

Figure 19: Extracting estimated R values

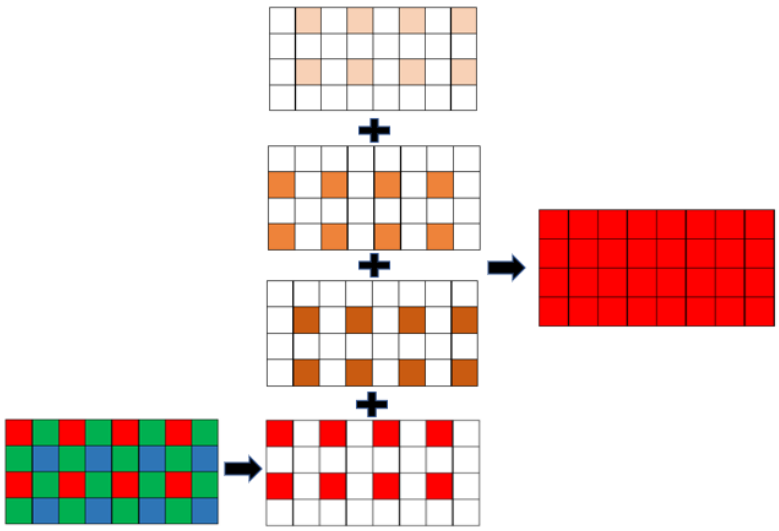

Figure 20: Final R Channel

Configuration Parameters

CFA has no configuration parameters and is always active in the pipeline.

Color Correction Matrix

To optimize color output in digital images, it is essential to address factors such as optical spectral properties (including lenses and filters), a variety of lighting sources (such as tungsten, fluorescent, and daylight), and the characteristics of the sensor's color that can lead to color inaccuracies in the captured image. The Color Correction Matrix (CCM) module plays a critical role in counteracting the effects of incorrect color representation and applies a color correction matrix to the image, ensuring a more precise display of colors as captured by the source imaging system.

Algorithm Explanation

The Color Correction Matrix (CCM) is a 3x3 matrix specifically designed to adjust the Red, Green, and Blue (RGB) color channels of an image. Each component within the matrix acts as a scaling factor, modifying the color values of each channel based on a calculated combination of the original values. This matrix is systematically applied to every pixel within the image, effectively transforming the color values in accordance with the given formula:

$$\begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} = CCM * \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

$$\begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} = \begin{bmatrix} RR & RG & RB \\ GR & GG & GB \\ BR & BG & BB \end{bmatrix} * \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Configuration Parameters

Table 11: CCM configuration parameters

Parameters	Details
is_enable	When enabled, the user given 3x3 CCM is applied to the 3D RGB image with rows sum to 1 convention: False: Disable True: Enable
corrected_red	Row 1 of CCM
corrected_green	Row 2 of CCM
corrected_blue	Row 3 of CCM

Gamma Correction

Gamma correction alters the input image by translating it to a nonlinear space. It matches the nonlinear response of display devices and broadens or compresses the dynamic range of images.

Algorithm Explanation

A look-up table is a common method for implementing gamma correction in hardware and is simple to modify and encode.

Configuration Parameters

Table 12: GC configuration parameters

Parameters	Details
is_enable	When enabled, applies tone mapping gamma using the look-up table: False: Disable True: Enable
gamma_lut_8	The look-up table for the gamma curve for 8-bit image
gamma_lut_10	The look-up table for the gamma curve for 10-bit image
gamma_lut_12	The look-up table for the gamma curve for 12-bit image
gamma_lut_14	The look-up table for the gamma curve for 14-bit image

Color Space Conversion

The color space conversion transforms the RGB color representation of an image to another color space. Different color spaces have been designed to accommodate varying proportions of chroma and luma. Hardware displays typically utilize the YUV color space. This choice is primarily rooted in the history of television technology, where black and white televisions relied solely on a single luminance channel. As color television emerged, the U and V channels were introduced to transmit color information, while the Y channel continued to convey luminance, thus ensuring backward compatibility with earlier black and white televisions.

Algorithm Explanation

In the model, the color space conversions are based on ITU-R standards. There are three ITU standards for YUV conversion.

1. BT.601 – for SDTV
2. BT.709 – for HDTV

We have implemented BT.601. BT.709 while BT.2020 will be implemented in the future.

BT.601 The conversion formula uses the ITU-R standard, multiplying the YUV_{mat} to the RGB image.

$$YUV_{mat} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix}$$

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = YUV_{mat} * \begin{bmatrix} R_{norm} \\ G_{norm} \\ B_{norm} \end{bmatrix}$$

The conversion requires a normalized RGB image, and the output YUV has a Y range from 0 to 1, whereas U and V range from -0.5 to 0.5. This conversion is not ideal for RTL implementation due to the multiplication involving floating-point numbers. Therefore, YUV_{mat} is converted into an integer matrix, also known as YCrCb, to enable digital conversion. The process for converting RGB to YC_rC_b using the YC_rC_{bmat} involves the following steps:

1. Multiply YUV_{mat} with 2^m, where m is the matrix precision bits to get YC_rC_{bmat} . The higher the bit value, the higher the precision, resulting in multiplication with larger bit depths. m can only be an even integer from 8 to 16. For the model, m is set to 8.

$$YC_rC_{bmat} = YUV_{mat} * \ 2^m$$

$$YC_rC_{bmat} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} * \ 2^m$$

$$YC_rC_{bmat} = \begin{bmatrix} 77 & 150 & 29 \\ 128 & -107 & -21 \\ -43 & -85 & 128 \end{bmatrix}$$

2. Multiply RGB with the YC_rC_{bmat} and add a digital offset to correct the black level of the YCrCb image. Divide the result of this matrix multiplication by 2^m to obtain a YCrCb image with the same bit depth as the input RGB image. Then, add a digital offset matrix to correct the black level of the YC_rC_b image.

$$\begin{bmatrix} Y \\ C_r \\ C_b \end{bmatrix} = \frac{\left(YC_rC_{bmat} * \begin{bmatrix} R \\ G \\ B \end{bmatrix} \right)}{2^m} + \begin{bmatrix} Y_{offset} \\ C_{roffset} \\ C_{boffset} \end{bmatrix}$$

3. This offset matrix depends on bit depth of YC_rC_b image n and it can be defined as:

$$\begin{bmatrix} Y_{offset} \\ C_{roffset} \\ C_{boffset} \end{bmatrix} = \begin{bmatrix} 2^{\frac{n}{2}} \\ 2^{n-1} \\ 2^{n-1} \end{bmatrix}$$

BT.709 BT.709 conversion is similar to BT.601 with different conversion matrix:

$$YUV_{mat} = \begin{bmatrix} 0.2126 & 0.7152 & 0.0722 \\ -0.1147 & -0.3854 & 0.5 \\ 0.5 & -0.4542 & -0.0458 \end{bmatrix}$$

This decimal matrix also known as analog matrix is also converted into digital matrix in the same way described in BT. 601 section following same steps to achieve the final YC_rC_b Image.

$$YC_rC_{bmat} = \begin{bmatrix} 47 & 157 & 16 \\ -26 & -86 & 112 \\ 112 & -102 & -10 \end{bmatrix}$$

Note: In all the module after the CSC module, YC_rC_b image is referred as YUV image.

Configuration Parameters

Table 13: CSC configuration parameters

Parameters	Details
conv_standard	Set standard to be used for conversion 1 : Bt.709 HD 2 : Bt.601/407

Local Dynamic Contrast Improvement

The Local Dynamic Contrast Improvement (LDCI) module applies Contrast Limited Adaptive Histogram Equalization (CLAHE) to the luma (Y) channel to improve local contrast. It runs after Color Space Conversion and before Sharpening; chroma (Cb/Cr) passes through unchanged.

Algorithm Explanation

LDCI works in two steps:

1. Per-tile histogram equalization
2. Bilinear blending across tile boundaries

   The frame is divided into *wind* × *wind* tiles (wind is window size). For each tile independently, a 256-bin histogram of its Y values is built, and a contrast-enhancement lookup table (LUT) is computed from that tile's own local histogram via CLAHE. The bin counts above clip_limit are clipped (to avoid over-amplifying noise in flat regions), and the clipped-off count is redistributed evenly across all 256 bins before the LUT is built from the cumulative distribution. Each tile ends up with its own LUT, tuned to its own local contrast.

   To avoid visible seams at tile boundaries, each output pixel is produced by bilinearly blending the LUTs of its neighbouring tiles. A tile in a frame corner uses only its own LUT (no blending), a tile on a frame edge blends with one neighbour, and an interior tile blends with its four surrounding tiles.

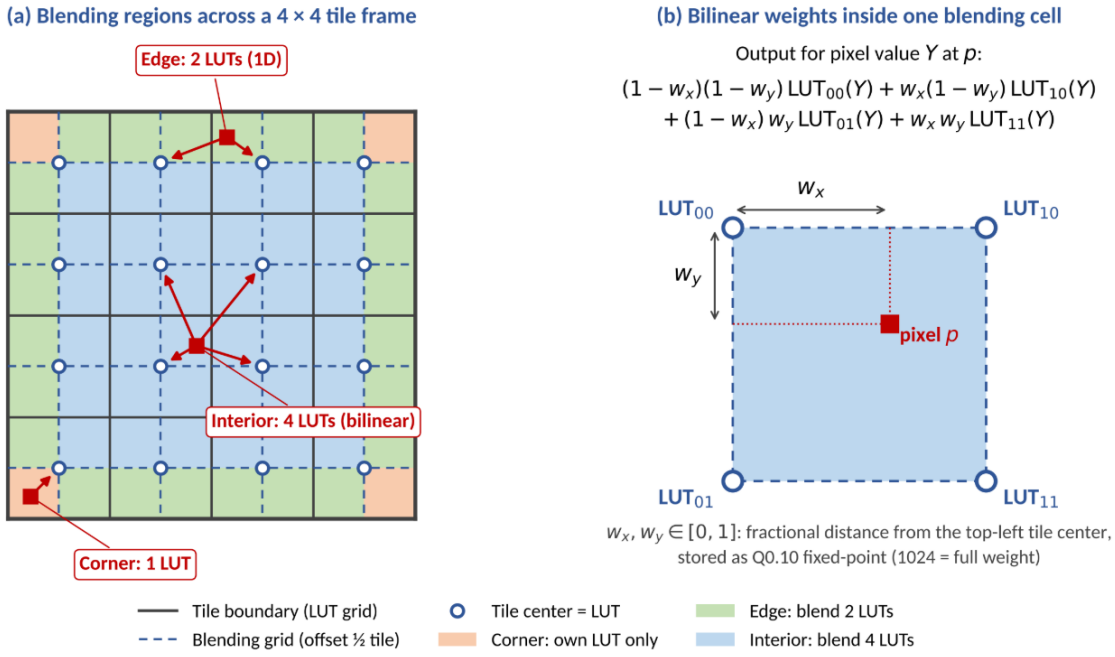

Figure 15: LDCI Tile boundary blending

Configuration Parameters

Table 14: LDCI configuration parameters

Parameters	Details
isEnable	When enabled, applies the LDCI to the image False: Disable True: Enable
clip_limit	Per-bin histogram clip threshold for a wind x wind tile (clamped to 8% of wind if set at or above wind)
wind	Tile size in pixels (each tile is wind x wind)

Sharpening

The Sharpen module amplifies the selected high-frequency band of the image in order to improve the edges and texture details of the image. The strength of the applied sharpening and the band selection are dependent on the 2DNR module in the pipeline to ensure high visual fidelity.

Algorithm Explanation

The Unsharp Masking uses the luma component of the image. In the first step, we convolve the luminance Y component of the input image with a Gaussian filter $G(\sigma)$ where σ is the standard deviation value. Next, we use this attained low-frequency component to extract the remaining higher frequencies H from the image in the following way:

$$H = Y - (G(\sigma) * Y)$$

The extracted high frequencies of then enhanced by the given strength k and then added back into the original luminance value in the following way to achieve the sharpened luma Y_{shrp}:

$$Y_{shrp} = Y + k \cdot H$$

Configuration Parameters

Table 15: Sharpening Configuration Parameters

Parameters	Details
isEnable	When enabled, applies the sharpening to the image False: Disable True: Enable
sharpen_sigma	[1, 10] – Parameter to define the Standard Deviation of the Gaussian Filter
sharpen_strength	[0.1, 2] – Parameter controls the sharpen strength applied on the high frequency components

2D Noise Reduction

A denoising tool, 2D Noise Reduction (2DNR), is incorporated into the system to minimize the impact of image noise, which deteriorates image quality. In conducting a literature review, the behavior of various spatial domain denoising methods was analyzed, along with the effects caused by adaptively chosen weights associated with them. Taking into account the hardware and computational constraints, we selected the Non-Local Mean (NLM) algorithm for denoising images.

Algorithm Explanation

Non-Local Means Filter The NLM algorithm is a spatial domain denoising technique. These types of methods can be further categorized as local and non-local filters. Natural images exhibit self-similarity, and the NLM filter leverages this characteristic. NLM denoises an image by taking the weighted average values of all similar pixels throughout the entire image.

The weights assigned to these similar pixels are computed based on the Euclidean distance between the intensities of the pixels. They are sorted in decreasing order to assign more weight to the most similar pixel in the image. This concept is founded on the idea that, for denoising a single pixel in an image, the weighted average is computed using all those pixels in the entire image, which share similar intensity or grey levels, and are close to each other based on their Euclidean distance.

By considering the spatial relationships between pixels and their intensities, the NLM filter effectively reduces noise while preserving important image details. However, it may be more computationally intensive than other denoising techniques due to its non-local approach.

Mathematically, we can represent this as:

$$I_{denoised}(x) = \frac{1}{C(x)} \sum_{y \in \Omega} I(y).W(x,y)$$

$$W(x,y) = \exp\left(-\frac{||I(x) - I(y)||^2}{h^2}\right)$$

$I_{denoised}(x)$, is the denoised intensity value at x.

$C(x)$, is the normalization factor to ensure that the weights sum up to 1.

Ω, represents the search window containing all pixels in the image.

$W(x,y)$, is the weight assigned to each pixel in the search window based on its similarity with the centered pixel.

h, is the strength parameter that controls the strength of the denoising effect by influencing the assignment of weights.

$I(y)$, represents all the pixels placed in the search window.

Configuration Parameters

Table 16: 2DNR configuration parameters

Parameters	Details
is_enable	When enabled, apply the non-local mean filtering: False: Disable True: Enable

window_size	Search window size for NLM filter. It should be odd 9 is set to avoid the computational cost.
wts	0-100] Strength parameter for NLM. The higher the value, the higher will be the smoothing or blurring Set to 5

Saturation Enhancement

Saturation Enhancement is a bit-exact fixed-point model of the saturation block. It runs in the YUV domain, after 2D Noise Reduction and before RGB Conversion, on the 8-bit YUV that Color Space Conversion produces. Each chroma sample (Cb and Cr) is scaled about the chroma pedestal using a fixed-point gain register; luma (Y) passes through unchanged. At the register's bit widths:

$$
\begin{aligned}
sub &= C - 128 && \text{(signed, 9 bit)} \\
mult &= sub \times \text{SAT_GAIN} && \text{(signed, 20 bit)} \\
shift &= mult \gg 8 && \text{(signed, 12 bit, arithmetic shift)} \\
add &= shift + 128 && \text{(signed, 12 bit)} \\
C' &= \text{clip}(add,\ 0,\ 255)
\end{aligned}
$$

SAT_GAIN is a 12-bit unsigned Q4.8 register (256 = a gain of 1.0), set from the saturation_gain configuration value truncated to the register's 1/256 steps. Gains above a computed maximum are refused: beyond that point the 12-bit add stage wraps, and a fully saturated chroma sample would flip to 0 instead of clipping to 255 - so every accepted configuration is clean by construction. This scaling previously ran inside Color Space Conversion, applied before the chroma pedestal was added; it is now its own module, and runs later - after noise reduction - so that saturation enhancement does not influence how 2D Noise Reduction perceives the image.

Configuration Parameters

Table 17: SE configuration parameters

Parameters	Details
is_enable	When enabled, saturation enhancement is applied to the chroma channels
saturation_gain	Positive float gain, truncated to the SAT_GAIN register (12-bit unsigned Q4.8, 1/256 steps). Values above the register's clean range are refused

RGB Conversion

RGB conversion module determines the output format of the model. If enabled, it performs inverse color space conversion from YUV to RGB based on conversion type specified in CSC module parameter otherwise the pipeline outputs a YUV image.

The conversion formula uses the ITU-R standard for BT-601 and BT_709, multiplying the RGB_{mat} to the YUV image.

$$RGB_{mat} = {YUV_{mat}}^{-1}$$

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = RGB_{mat} * \begin{bmatrix} Y \\ U \\ V \end{bmatrix}$$

RGB_{mat} matrix is a decimal; therefore, it is converted into an integer matrix using the following steps:

1. Multiply YUV_{mat} with 2^m, where m is the matrix precision bits to get YC_rC_{bmat} . The higher the bit value, the higher the precision, resulting in multiplication with larger bit depth m can only be an even integer from 8 to 16. For the model, m is set to 8.

$$digital_RGB_{mat} = RGB_{mat} * 2^m$$

$$digital_RGB_{mat} = \begin{bmatrix} 64 & 0 & 90 \\ 64 & -22 & -46 \\ 64 & 113 & 0 \end{bmatrix}$$

2. Subtract the digital offset and multiply YUV image with the $digital_RGB_{mat}$. Divide the result of this matrix multiplication by 2^m to obtain a RGB image with the same bit depth as the input RGB image.

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \frac{digital_RGB_{mat} * \left(\begin{bmatrix} Y \\ U \\ V \end{bmatrix} - \begin{bmatrix} Y_{offset} \\ C_{roffset} \\ C_{boffset} \end{bmatrix} \right)}{2^m}$$

3. This offset matrix depends on bit depth of YUV image bit depth $n = 8$ and it can be defined as:

$$\begin{bmatrix} Y_{offset} \\ C_{roffset} \\ C_{boffset} \end{bmatrix} = \begin{bmatrix} 2^{\frac{n}{2}} \\ 2^{n-1} \\ 2^{n-1} \end{bmatrix}$$

Configuration Parameters

Table 18: RGBC configuration parameters

Parameters	Details
is_enable	When enabled, converts output YUV image to RGB False : Disable – Output image format is YUV True : Enable – Output Image format is RGB

Invalid Region Crop

The Invalid Region Crop (IRC) module functions in unison with the scaling module to ensure images are cropped to a valid size, specifically 1920x1080 or 1920x1440. This step is essential for the utilization of a hardware-friendly scaling approach, which supports only the specified input sizes. It's critical to note that if the IRC module is deactivated, the is_hardware flag in the Scale module should also be switched off. This adjustment maintains the consistent and error-free execution of the scaling operations.

Configuration Parameters

Table 19: IRC configuration parameters

Parameters	Details
is_enable	When enabled, scales down the input image. False: Disable True: Enable
crop_to_size	Only two permissible values to set crop size: 1 : crops the image to size 1920x1080 2 : crops the image to size 1920x1440
height_start_idx	Starting height-index for crop
width_start_idx	Starting width-index for crop

Scale

The Scale block is implemented as one of the final blocks in the ISP pipeline to downscale a full-resolution image to a supported resolution/size per the user's requirements. The Scale block is generally implemented in IP camera ISPs to create multiple channels/streams for storage and transmission in surveillance cameras.

Scaling only preserves the aspect ratio (the ratio between image height and width $\frac{h}{w}$) when height and width are scaled by the same factor $\frac{h*c}{w*c} = \frac{h}{w}$. Cropping does not preserve the aspect ratio even if same amount is cropped from both height and width $\frac{h-c}{w-c} \neq \frac{h}{w}$.

Up/Down scaling height and width at the same time is equivalent to scaling the height first and width second or vice versa. For example, downscaling with Nearest Neighbor method using a square 2×2 window is equal to downscaling by 2×1 window followed by downscaling with 1×2 window.

Algorithm Explanation

- Input image: 3D array (RGB or YUV image) of size 1920×1080 or 1920×1440.
- Output image: Downscaled 3D array (RGB or YUV image). The valid output sizes corresponding to each input size are mentioned in Table 20.

The image is scaled using the following steps:

1. Down-scale the image by the integer scaling factor using the nearest neighbor method.
2. Crop the image to required output-size if needed.
3. The algorithm scales each channel one by one using the same steps.

The table below shows the hand-picked scaling factors and crop values that have been used in the design. These values are hardcoded in the code for each input size using classes.

Table 20: Scaling – Valid Output Sizes

Input Size	**Output size**	**Downscale factor**		**Crop value**	
		width	**Height**	**width**	**Height**
1920 × 1080	640 × 480	3	2	–	60
1920 × 1080	640 × 360	3	3	–	–
1920 × 1440	960 × 720	2	2	–	–
1920 × 1440	640 × 480	3	3	–	–

Configuration Parameters

Table 21: Scale configuration parameters

Parameters	**Details**
is_enable	When enabled, scales down the input image False: Disable True: Enable
new_width	Downscaled width of the output image
new_height	Downscaled height of the output image

YUV Format - 444-422

The YUV conversion format is a way of subsampling YUV images to save the bandwidth. Subsampling of YUV is possible because the human eye is less sensitive to color differences compared to luminance differences, allowing for subsampling of the C_rC_b channel without significantly affecting the output. That is why no subsampling is done for the Y channel.

Algorithm Explanation

In YUV format, the Y, U, and V components are stored in a single array, known as the packed format. Pixels are organized into macro pixel groups, with the layout depending on the YUV format. Each YUV format described has an assigned FOURCC code, which is a 32-bit unsigned integer created by concatenating four ASCII characters:

- Y : Luma
- U : Blue-Difference
- V : Red-Difference
- A : Transparency

YUV formats implemented in the model are

1. 4:4:4
    a. AYUV
2. 4:2:2
    a. YUYV
    b. UYVY

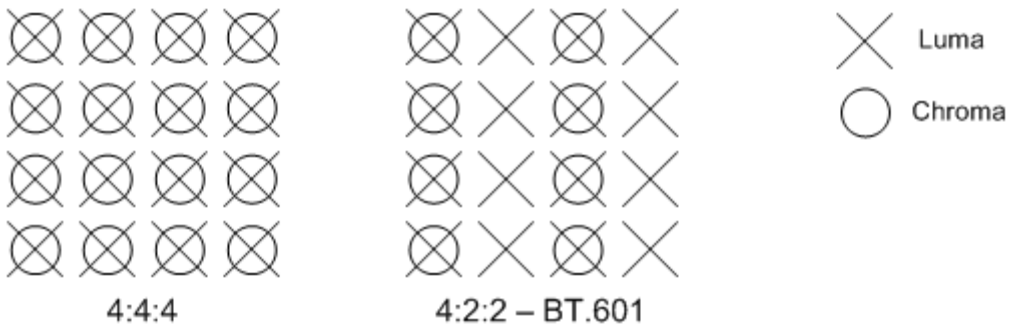

Figure 21: Pictorial representation of 4:4:4 and 4:2:2

4:4:4 Format In this format, no subsampling is done, and YUV is saved in the following manner:

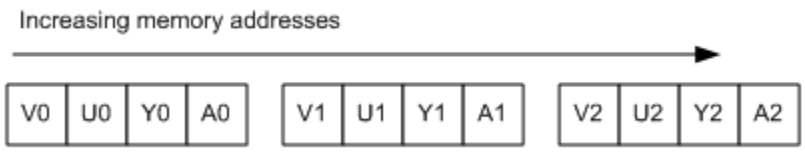

Figure 22: Memory view of bytes for packed format

The recommended FOURCC format for 4:4:4 is shown above. In FOURCC, each memory element is 32 bits with 4 bytes, each representing either Y, U, or V based on the chosen formatting. In the model, a simple 4:4:4 format is implemented, where each memory chunk has 3 bytes of Y, U, or V. Since there is no information regarding the alpha channel yet, it is not considered in this conversion.

Pixel Number	Pixel Values
0	U0Y0V0
1	U1Y1V1
2	U2Y2V2
...	...

Figure 23: 4:4:4 format

4:2:2 Format In this format, every two pixels share U and V. This is achieved by using the same U and V values for each pair of pixels. For example, the same U and V values are used for Y1 and Y2, and the same U and V values are used for Y3 and Y4.

Pixel Number	Pixel Values
0	0Y0V0
1	U0Y1V0
2	U2Y2V2
3	U2Y3V2
4	U4Y4V4
...	...

Figure 24: 4:2:2 format

In the FOURCC format, the above representation is saved as follows:

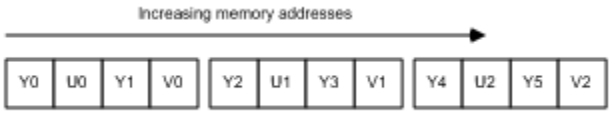

Figure 25: Memory view of 4:2:2

The first four bytes represent two pixels. Both pixels have the same U and V values but different Y values. Note that U and V in the Figure 25 are subsampled entries of the corresponding U and V channels.

In the model, subsampling is done horizontally for each row of the U and V channels, and the YUYV format is implemented.

Configuration Parameters

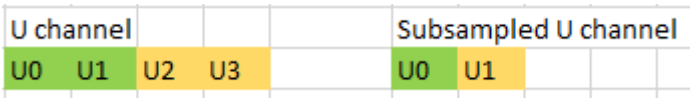

Figure 26: Subsample entries for 4:2:2

Table 22: YUV Formats configuration parameters

Parameters	**Details**
is_enable	Will enable or disable this module: False: Disable True: Enable
conv_type	Set conversion format of YC_rC_b to YUV. Can only be two values: - 444 - 422

Pipeline Results

Here are the results of this pipeline compared with a market-competitive ISP. The model outputs are displayed on the right, with the underlying ground truths on the left.

Ground Truth **Infinite-ISP Reference Model**

Figure 27: Pipeline Results

A comparison of the above results based on the PSNR and SSIM image quality metrics is given below. PSNR is computed on the full-resolution RGB output; SSIM follows the Wang reference implementation (luma, with automatic downsampling). The results are generated with the default configuration and the 3A control (Auto Exposure and Auto White Balance) enabled.

Table 23: IQ metrics of the reference-model output against ground truth

Image	**PSNR (dB)**	**SSIM**
Indoor1	22.3860	0.9486
Outdoor1	17.4022	0.8712
Outdoor2	18.4249	0.8063
Outdoor3	16.4199	0.7191
Outdoor4	23.2950	0.9456

References

1. Measurement for optoelectronic conversion functions (OECFs) of the digital still-picture camera - NASA/ADS (harvard.edu)
2. https://www.imatest.com/?s=oecf
3. https://www.imatest.com/docs/esfriso_instructions/
4. https://slideplayer.com/slide/5759755/
5. ISO - ISO 14524:2009 - Photography — Electronic still-picture cameras — Methods for measuring optoelectronic conversion functions (OECFs)
6. (PDF) Linearisation of RGB Camera Responses for Quantitative Image Analysis of Visible and UV Photography: A Comparison of Two Techniques (researchgate.net)
7. Linearization in detail - three different ways to linearize images (image-engineering.de)
8. https://patentimages.storage.googleapis.com/f9/11/65/a2b66f52c6dbd4/US8538199.pdf
9. https://static.aminer.org/pdf/PDF/000/319/504/large_scale_infographic_image_downsizing.pdf
10. https://www.ipol.im/pub/art/2011/g_mhcd/article.pdf
11. https://www.ipol.im/pub/art/2011/bcm_nlm/article.pdf
12. https://www.itu.int/rec/R-REC-BT.709-6-201506-I/en
13. https://www.flir.com/support-center/iis/machine-vision/knowledge-base/understanding-yuv-data-formats/
14. https://en.wikipedia.org/wiki/Chroma_subsampling
15. https://www.cs.auckland.ac.nz/courses/compsci773s1c/lectures/YuY2_files/intro.htm#YV12
16. https://learn.microsoft.com/en-us/windows/win32/medfound/recommended-8-bit-yuv-formats-for-video-rendering
17. https://sci-hub.hkvisa.net/10.1109/ICAIIS49377.2020.9194921
18. https://sci-hub.hkvisa.net/10.1109/apccas.2012.6419046
19. https://www.atlantis-press.com/article/25875811.pdf
20. https://www.sciencedirect.com/science/article/abs/pii/0016003280900587
21. https://library.imaging.org/admin/apis/public/api/ist/website/downloadArticle/cic/12/1/art00008
22. https://opg.optica.org/josaa/viewmedia.cfm?uri=josaa-31-5-1049&seq=0
23. https://www.hindawi.com/journals/tswj/2014/979081/
24. https://www.semanticscholar.org/paper/Contrast-Limited-Adaptive-Histogram-Equalization-Zuiderveld/726c95fa3bd47401befc7513b6e52d1c806f26af.
25. https://web.archive.org/web/20120113220509/http://radonc.ucsf.edu/research_group/jpouliot/tutorial/HU/Lesson7.htm
26. https://github.com/cruxopen/openISP
27. https://github.com/QiuJueqin/fast-openISP